Day 15: Virtual Functions & Polymorphism

📚 Learning Objectives

By the end of this day, you should understand:

- What are virtual functions and why they're needed
- Runtime polymorphism vs Compile-time polymorphism
- Virtual destructors and their importance
- Pure virtual functions and abstract classes
- VTable and VPtr internals
- The `override` keyword

📖 Part 1: Virtual Functions

Concept Explanation

In C++, there are two types of polymorphism:

1. **Compile-time Polymorphism (Early Binding)**
   - Function overloading
   - Operator overloading
   - Resolved at compile time
2. **Runtime Polymorphism (Late Binding)**
   - Achieved through **virtual functions**
   - Resolved at runtime using VTable

Why Virtual Functions?

Consider this scenario:

```cpp
class Shape {
public:
    void draw() {
        cout << "Drawing a shape\n";
    }
};

class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing a circle\n";
    }
};

class Rectangle : public Shape {
public:
    void draw() {
        cout << "Drawing a rectangle\n";
    }
};

int main() {
    Shape* s1 = new Circle();
    Shape* s2 = new Rectangle();

    s1->draw();  // What gets called?
    s2->draw();  // What gets called?

    delete s1;
    delete s2;
}
```

Without `virtual` keyword:

- `s1->draw()` calls `Shape::draw()` ❌ (not `Circle::draw()`)
- `s2->draw()` calls `Shape::draw()` ❌ (not `Rectangle::draw()`)

This happens because the compiler uses **static binding** (early binding) - it decides which function to call based on the **pointer type** (`Shape*`), not the **object type** (`Circle` or `Rectangle`).

The Solution: Virtual Functions

```
class Shape {
public:
    virtual void draw() {  // Add 'virtual' keyword
        cout << "Drawing a shape\n";
    }
};
```

With `virtual` keyword:

- `s1->draw()` calls `Circle::draw()` ✅
- `s2->draw()` calls `Rectangle::draw()` ✅

Now the compiler uses **dynamic binding** (late binding) - it decides at runtime based on the actual object type.

How It Works Internally (VTable & VPtr)

When you use virtual functions:

1. **VTable (Virtual Table)**: Compiler creates a table of function pointers for each class with virtual functions
2. **VPtr (Virtual Pointer)**: Each object gets a hidden pointer to its class's VTable

```
Memory Layout:

Circle Object:          Circle VTable:
┌──────────────┐        ┌──────────────────┐
│ VPtr ────────┼────────► &Circle::draw    │
├──────────────┤        │ &Circle::~Circle │
│ Circle data │        └──────────────────┘
└──────────────┘
```

When you call `s1->draw()` :

1. Access object's VPtr
2. Follow VPtr to VTable
3. Get function pointer from VTable
4. Call the function

The `override` Keyword (C++11)

Always use `override` in derived classes:

```
class Circle : public Shape {
public:
    void draw() override {  // Compiler checks if base has virtual draw()
        cout << "Drawing a circle\n";
    }
};
```

Benefits:

- Compiler catches errors if base function signature changes
- Makes code more readable and intentional
- Prevents accidental function hiding

📖 Part 2: Virtual Destructors

The Problem

```
class Base {
public:
    Base() { cout << "Base constructor\n"; }
    ~Base() { cout << "Base destructor\n"; }  // Non-virtual destructor
};

class Derived : public Base {
public:
    Derived() {
        cout << "Derived constructor\n";
        data = new int[1000];  // Allocating memory
    }
    ~Derived() {
        cout << "Derived destructor\n";
        delete[] data;  // Cleanup
    }
private:
    int* data;
};

int main() {
    Base* obj = new Derived();
    delete obj;  // Which destructor is called?
}
```

Output:

```
Base constructor
Derived constructor
Base destructor    ← Only base destructor called!
```

Problem: `Derived` destructor is NEVER called! This causes:

- Memory leak (data is never deleted)
- Resource leak (files, connections not closed)

The Solution: Virtual Destructor

```
class Base {
public:
    virtual ~Base() {  // Make destructor virtual
        cout << "Base destructor\n";
    }
};
```

Now the output:

```
Base constructor
Derived constructor
Derived destructor   ← Derived destructor called first
Base destructor      ← Then base destructor
```

Rule: Always use virtual destructor in base class if:

- ✅ Class has at least one virtual function
- ✅ You intend to delete derived objects through base pointers
- ✅ Class is meant to be inherited

📖 Part 3: Pure Virtual Functions & Abstract Classes

Pure Virtual Functions

A **pure virtual function** is a virtual function with no implementation in the base class:

```
class Animal {
public:
    virtual void makeSound() = 0;  // Pure virtual function
};
```

The `= 0` means "this function MUST be implemented by derived classes".

Abstract Classes

A class with **at least one pure virtual function** is called an **Abstract Class**.

Properties:

- ❌ Cannot create objects of abstract class
- ✅ Can create pointers/references to abstract class
- ✅ Derived classes MUST implement all pure virtual functions

```
Animal a;           // ERROR: Cannot create object of abstract class
Animal* ptr;        // OK: Can create pointer

class Dog : public Animal {
public:
    void makeSound() override {  // MUST implement
        cout << "Woof!\n";
    }
};

Dog d;            // OK: Dog is not abstract
Animal* a = &d;   // OK: Polymorphism
```

Interface Design

Abstract classes are used to define **interfaces** (contracts):

```
class PaymentProcessor {
public:
    virtual void processPayment(double amount) = 0;
    virtual bool validate() = 0;
    virtual ~PaymentProcessor() = 0;
};

PaymentProcessor::~PaymentProcessor() {}  // Pure virtual destructor needs implementation

class CreditCardProcessor : public PaymentProcessor {
public:
    void processPayment(double amount) override { /* ... */ }
    bool validate() override { /* ... */ }
    ~CreditCardProcessor() override { /* ... */ }
};
```

📖 Part 4: Important Interview Concepts

1. Early Binding vs Late Binding

Early Binding	Late Binding
Compile-time	Runtime
Non-virtual functions	Virtual functions
Faster	Slightly slower (VTable lookup)
Based on pointer type	Based on object type

2. Override vs Hide

```
class Base {
public:
    virtual void show(int x) { cout << "Base::show(int)\n"; }
    virtual void show(double d) { cout << "Base::show(double)\n"; }
};

class Derived : public Base {
public:
    void show(int x) override {  // Overrides Base::show(int)
        cout << "Derived::show(int)\n";
    }

    void show(string s) {  // HIDES all Base::show functions!
        cout << "Derived::show(string)\n";
    }
};

Derived d;
d.show(10);        // Calls Derived::show(int)
d.show(5.5);       // ERROR: Base::show(double) is hidden!
d.show("hello");   // Calls Derived::show(string)
```

Solution: Use `using` to bring base functions into scope:

```
class Derived : public Base {
public:
    using Base::show;  // Bring all Base::show into scope

    void show(int x) override { /* ... */ }
    void show(string s) { /* ... */ }
};
```

3. Object Size with Virtual Functions

```
class Empty {};                          // sizeof = 1 byte
class WithVirtual {                      // sizeof = 8 bytes (VPtr)
    virtual void func() {}
};
class MultipleVirtual {                  // sizeof = 16 bytes (VPtr + int)
    virtual void func1() {}
    virtual void func2() {}
    int data;
};
```

Key Point: Adding more virtual functions doesn't increase object size (VPtr is same, VTable is per-class).

4. Can Constructors be Virtual?

NO! Constructors cannot be virtual because:

- VPtr is initialized during construction
- Virtual mechanism requires VPtr to already exist
- Object type is known at construction time

5. Can Destructors be Pure Virtual?

YES! But they must have an implementation:

```
class Abstract {
public:
    virtual ~Abstract() = 0;  // Pure virtual destructor
};

Abstract::~Abstract() {}  // Must provide implementation
```

✍️ Practice Questions

Question 1: Predict the Output

```
class A {
public:
    void show() { cout << "A::show\n"; }
};

class B : public A {
public:
    void show() { cout << "B::show\n"; }
};

int main() {
    A* ptr = new B();
    ptr->show();
    delete ptr;
}
```

Q1.1: What is the output?

Q1.2: Why does this happen?

Q1.3: How would you fix it to call B::show()?

Question 2: Virtual Destructor Bug

```
class Base {
public:
    Base() { buffer = new char[1024]; }
    ~Base() { cout << "Base destroyed\n"; }
private:
    char* buffer;
};

class Derived : public Base {
public:
    Derived() { data = new int[500]; }
    ~Derived() {
        cout << "Derived destroyed\n";
        delete[] data;
    }
private:
    int* data;
};

int main() {
    Base* obj = new Derived();
    delete obj;
    return 0;
}
```

Q2.1: What's wrong with this code?
Q2.2: What will be the output?
Q2.3: What memory issues exist?
Q2.4: Fix the code

Question 3: Abstract Class Implementation

Create a complete working program:

```
// Abstract class
class Shape {
protected:
    string color;

public:
    Shape(string c) : color(c) {}

    // Pure virtual functions (implement these in derived classes)
    virtual double area() = 0;
    virtual double perimeter() = 0;
    virtual string getName() = 0;

    // Virtual function with default implementation
    virtual void describe() {
        cout << "A " << color << " shape\n";
    }

    // Virtual destructor
    virtual ~Shape() = 0;
};

// TODO: Implement Shape::~Shape()

// TODO: Create Circle class that inherits from Shape
// - Private member: radius
// - Constructor: Circle(string color, double radius)
// - Implement: area(), perimeter(), getName(), describe()
// - area = π * r², perimeter = 2 * π * r

// TODO: Create Rectangle class that inherits from Shape
// - Private members: width, height
// - Constructor: Rectangle(string color, double w, double h)
// - Implement: area(), perimeter(), getName(), describe()
// - area = w * h, perimeter = 2 * (w + h)

// TODO: Create Triangle class that inherits from Shape
// - Private members: side1, side2, side3
// - Constructor: Triangle(string color, double s1, double s2, double s3)
// - Implement: area() using Heron's formula, perimeter(), getName(), describe()

int main() {
    // Create array of Shape pointers
```

```
    Shape* shapes[4];

    // TODO: Initialize with Circle, Rectangle, Triangle, Circle

    // TODO: Loop through and print: name, color, area, perimeter

    // TODO: Cleanup (delete all shapes)

    return 0;
}
```

Question 4: VTable Analysis

```
class Base {
public:
    virtual void func1() { cout << "Base::func1\n"; }
    virtual void func2() { cout << "Base::func2\n"; }
    virtual void func3() { cout << "Base::func3\n"; }
    int baseData;
};

class Derived : public Base {
public:
    void func1() override { cout << "Derived::func1\n"; }
    void func2() override { cout << "Derived::func2\n"; }
    // func3 not overridden
    void func4() { cout << "Derived::func4\n"; }
    int derivedData;
};
```

Q4.1: Draw the VTable for Base class

Q4.2: Draw the VTable for Derived class

Q4.3: What is sizeof(Base)? (Assume 64-bit system, int = 4 bytes)

Q4.4: What is sizeof(Derived)?

Q4.5: If we add 5 more virtual functions to Base, how does object size change?

Question 5: Fix the Code

The following code has multiple issues. Find and fix all of them:

```cpp
class Animal {
public:
    virtual void eat() {
        cout << "Animal eating\n";
    }

    void sleep() {
        cout << "Animal sleeping\n";
    }
};

class Dog : public Animal {
public:
    void eat() {
        cout << "Dog eating\n";
    }

    void sleep() {
        cout << "Dog sleeping\n";
    }

    void bark() {
        cout << "Woof!\n";
    }
};

class Cat : public Animal {
public:
    void eat(int foodAmount) {  // Different signature
        cout << "Cat eating " << foodAmount << " grams\n";
    }
};

int main() {
    Animal* animals[3];
    animals[0] = new Dog();
    animals[1] = new Cat();
    animals[2] = new Animal();

    for (int i = 0; i < 3; i++) {
        animals[i]->eat();
        animals[i]->sleep();
    }
```

```
    // Cleanup
    for (int i = 0; i < 3; i++) {
        delete animals[i];
    }

    return 0;
}
```

Issues to find:

1. Missing virtual destructor
2. Cat::eat() hides instead of overrides
3. Cannot instantiate Animal if we make it abstract
4. Dog::sleep() should be virtual in base

Question 6: Design Challenge - Payment System

Design a payment processing system:

```
// TODO: Create abstract class PaymentMethod
// Requirements:
// - Protected member: accountId (string)
// - Pure virtual: processPayment(double amount)
// - Pure virtual: getPaymentType()
// - Virtual: refund(double amount) with default implementation
// - Virtual destructor

// TODO: Create CreditCard class
// - Additional members: cardNumber, cvv, expiryDate
// - processPayment: Print "Processing $X on CreditCard ending XXXX"
// - getPaymentType: Return "Credit Card"

// TODO: Create PayPal class
// - Additional member: email
// - processPayment: Print "Processing $X via PayPal (email)"
// - getPaymentType: Return "PayPal"

// TODO: Create Crypto class
// - Additional members: walletAddress, coinType
// - processPayment: Print "Processing $X in CoinType to wallet"
// - getPaymentType: Return "Cryptocurrency"
// - Override refund to print crypto-specific message

int main() {
    // Create vector of PaymentMethod pointers
    // Add 2 CreditCard, 2 PayPal, 1 Crypto
    // Process payments of different amounts
    // Refund some payments
    // Cleanup
}
```

Question 7: Interview Questions

Answer these without coding:

Q7.1: What is the difference between virtual function and pure virtual function?

Q7.2: Can a constructor be virtual? Why or why not?

Q7.3: What happens if a derived class doesn't implement a pure virtual function?

Q7.4: Why is VTable per-class and not per-object?

Q7.5: What is the size of an empty class vs a class with only virtual functions?

Q7.6: Can a static member function be virtual? Why or why not?

Q7.7: What is the order of constructor/destructor calls in inheritance with virtual functions?

Q7.8: Explain the difference between `override` and `redefine` (hiding).

📝 Summary Checklist

After completing this day, you should be able to:

- ☐ Explain what virtual functions are and why they're needed
- ☐ Differentiate between early binding and late binding
- ☐ Explain VTable and VPtr internals
- ☐ Write code with proper virtual destructors
- ☐ Create and use abstract classes with pure virtual functions
- ☐ Use the `override` keyword correctly
- ☐ Identify and fix virtual function related bugs
- ☐ Answer common interview questions on polymorphism

🔗 Additional Resources

- LearnCpp - Virtual Functions
- GeeksforGeeks - Virtual Functions
- cppreference - Virtual functions

➡️ Next: Day 16 - Copy Constructor & Rule of 3/5

After mastering virtual functions, you'll learn about:

- Copy constructors
- Copy assignment operators
- Deep copy vs Shallow copy
- Rule of 3, Rule of 5
- Resource management